UNIDAD 8 / SEMANAS 12 Y 13

Bases de datos móviles e integración

Material teórico · Una historia de Food Store en once paradas

FOOD STORE | PROYECTO INTEGRADOR

Asignatura	Base de Datos II · Tecnicatura Universitaria en Programación
Unidad	8 — Bases de Datos para Dispositivos Móviles e Integración
Semanas	12 (Móviles) y 13 (Integración)
Motores	PostgreSQL 16+ · SQLite 3.37+ · MongoDB (Unidad 7)
Herramientas de IA	Kiro (especificación) · OpenCode (generación y revisión)
Se evalúa en	TP N.° 8 (última entrega antes del parcial) · segundo parcial (semana 14)

Ciclo lectivo 2026 · Cátedra de Base de Datos II

Cómo está armado este material

Esta unidad tiene varios nombres nuevos: instantánea, lápida, cola de envío, UUID, idempotencia… Para que no queden como una lista suelta, **todo el material cuenta una sola historia**: un pedido que se hace en una feria donde no hay señal. Cada tema aparece justo cuando la historia lo necesita, y cada parada termina con la pregunta que responde la siguiente.

Cada parada tiene	Para qué sirve
La pregunta de esta parada	Qué problema vamos a resolver
En el food truck	Qué está pasando en la historia
La explicación	La idea en palabras simples, con un ejemplo de Food Store
Se conecta con lo que ya viste	Qué tema de unidades anteriores vuelve a aparecer
En resumen	Lo que te tenés que llevar y la pregunta que abre la parada siguiente

Al final hay un laboratorio, un repaso de una página, preguntas de autoevaluación con respuestas, un glosario y la bibliografía. Todo el SQL de este apunte fue probado en PostgreSQL 16 y SQLite 3.45.

Mapa de la Unidad 8: once paradas de una misma historia

Cada parada responde la pregunta que deja abierta la anterior. A la derecha, con qué se conecta.



Mapa de la unidad: once paradas de una misma historia.

Parada 0 · El problema: vender sin señal

> **La pregunta de esta parada**
> ¿Qué cambia cuando la base de datos no está siempre al alcance?

> **En el food truck**
> Sábado, feria de comidas. Food Store lleva un food truck con una tablet. La señal va y viene. Ana pide **1 muzzarella y 2 empanadas**. El vendedor no puede decirle "espere que vuelva Internet": tiene que poder mostrarle los precios y anotar el pedido igual.

Hasta ahora, en toda la cursada, había **una sola base**: PostgreSQL. Todos se conectaban a ella y ella decidía todo. En la feria, la tablet no siempre puede conectarse. Eso trae tres riesgos:

- **Perder** un pedido (se anotó, pero nunca llegó al servidor).
- **Duplicar** un pedido (llegó dos veces).
- **Vender lo que no hay** (la tablet creía que quedaba stock).

Toda la unidad se ordena alrededor de una regla, la **regla de oro**:

> **Regla de oro de la unidad**
> **El móvil propone, el servidor dispone.** La tablet consulta y prepara borradores; PostgreSQL es el único que decide el precio, el stock y si una venta existe.

Qué vas a poder hacer al terminar

1. Explicar qué es una base embebida (SQLite) y en qué se diferencia de PostgreSQL.
2. Decidir qué datos viajan al dispositivo y cuáles no, según las limitaciones del entorno móvil.
3. Diseñar una base SQLite para consultar productos y armar borradores sin conexión.
4. Explicar cómo se sincroniza con el servidor y cómo se resuelven los conflictos de precio, de stock y los reenvíos.
5. Integrar todo lo hecho en la cursada (PostgreSQL, MongoDB y SQLite) en una sola capa de datos.
6. Revisar los scripts con ayuda de IA y defender cada decisión.

Cómo se organizan las dos semanas

Semana	Paradas	En el aula	Lo que te llevás para el TP
12 · Móviles	1 a 8	Teoría + laboratorio: base SQLite del dominio con sincronización simulada y conflictos resueltos	Esquema local, datos de prueba y simulación que se puede repetir
13 · Integración	9 y 10	Teoría + taller: revisión con IA de todos los scripts y ensayo de defensa	Capa de datos completa, con specs, scripts versionados, plan de mantenimiento y bitácora de IA

Alcance: no vamos a programar una app de Android ni de iPhone. La base “de la tablet” es un archivo SQLite en tu computadora, y la sincronización es una **simulación** que se puede verificar, como pide el programa de la materia.

En resumen

- Ahora hay dos bases: la del servidor (PostgreSQL) y la de la tablet.
- El riesgo es perder, duplicar o vender lo que no hay.
- Regla de oro: el móvil propone, el servidor dispone.

Pregunta que queda abierta → ¿Dónde anota el vendedor si no hay servidor? → Parada 1

SEMANA 12 · LA HISTORIA DE UN PEDIDO SIN SEÑAL

Parada 1 · Una base dentro de la tablet: SQLite

> **La pregunta de esta parada**
>
> ¿Dónde guardamos los datos cuando no hay conexión con el servidor?

> **En el food truck**
>
> El vendedor necesita una **libreta**: algo que esté siempre en la tablet, aunque no haya señal. Esa libreta es una base de datos **embebida**.

Una **base embebida** es una base de datos que vive *dentro* de la aplicación. No hay un servidor aparte: toda la base es **un archivo** en el dispositivo. SQLite es la más usada del mundo: está en los teléfonos Android y iPhone, en los navegadores y en Python. Su documentación lo resume así: SQLite no compite con las bases cliente-servidor, compite con abrir un archivo.

	PostgreSQL (la oficina central)	SQLite (la libreta del vendedor)
Dónde está	En un servidor; nos conectamos por red	Dentro de la app; es un archivo
Sin Internet	No se puede usar	Funciona con lo que tenga guardado
Usuarios y permisos	Sí: roles y `GRANT` (Unidad 5)	No tiene: lo protege el sistema operativo
Escrituras al mismo tiempo	Muchas a la vez	Una por vez
En Food Store	Decide precio, stock y ventas	Muestra el catálogo y guarda borradores

SQLite **es una base relacional**: tiene tablas, SQL, claves primarias, claves foráneas y transacciones. No es NoSQL. Lo que cambia es dónde vive y algunas costumbres que hay que conocer:

Tres cosas de SQLite que tenés que saber

1. **Las claves foráneas vienen apagadas.** Hay que escribir `PRAGMA foreign_keys = ON;` cada vez que se abre la base. Si no, SQLite deja guardar un ítem de un producto que no existe.
2. **Los tipos son flexibles.** En una tabla común, una columna `INTEGER` acepta un texto. Para que controle los tipos se declara la tabla como `STRICT` (desde la versión 3.37). Tampoco tiene tipo booleano (usa 0 y 1) ni tipo fecha (usa texto).
3. **Escribe una cosa por vez.** Muchos pueden leer, pero solo uno escribe a la vez. Por eso las operaciones tienen que ser cortas.

¿Y en el celular de verdad? Las herramientas para apps móviles usan SQLite por debajo: **Room** en Android y **Core Data** en iPhone. Por eso aprender SQLite directo sirve para cualquiera de ellas.

> **Se conecta con lo que ya viste**
>
> En la **Unidad 7** viste que existen otras bases además de la relacional del servidor. SQLite es otra base, pero relacional: lo nuevo no es el modelo de datos, sino **dónde vive**.

En resumen

- Una base embebida vive dentro de la app y es un archivo.
- SQLite es relacional: usa SQL, claves y transacciones.
- Ojo con tres cosas: activar las claves foráneas, usar `STRICT` y escribir de a una operación.

Pregunta que queda abierta → ¿Qué anotamos en la libreta? ¿Copiamos todo el servidor? → Parada 2

Parada 2 · Qué datos viajan a la tablet

La pregunta de esta parada

¿Qué datos guardamos en el dispositivo y cuáles no?

En el food truck

El vendedor necesita **la lista de precios**, no **el archivo de clientes**. Si la tablet se pierde en la feria, lo que tenga guardado puede terminar en manos de cualquiera.

No se copia la base del servidor entera. Se elige según dos cosas: **qué necesita el vendedor para trabajar** y **qué limitaciones tiene un teléfono**.

Limitación del teléfono	Qué decidimos
La señal va y viene	Lo necesario para vender se guarda en la tablet; lo que hay que enviar espera en una cola
Sin red no hay quien numere los pedidos	Los borradores usan un identificador propio (UUID, parada 4)
La hora de cada teléfono puede estar mal	La hora del teléfono nunca decide qué dato vale
Poco espacio y batería	Solo una parte del catálogo; nada de reportes ni historia de ventas
La app se puede cerrar de golpe	Cada operación se graba entera o no se graba (parada 4)
El teléfono se puede perder	Nada de contraseñas, mails, celulares ni pagos
Hay varias tablets vendiendo	El stock lo decide solo el servidor

Dato	¿Viaja?	Por qué
Producto: id, nombre, precio, stock, disponible, dado de baja	Sí, como copia	Para mostrar el catálogo sin red (el stock es solo orientativo)
Borradores de pedido	Se crean en la tablet	Para anotar sin red; todavía no son ventas
Pedidos confirmados	No	Solo el servidor confirma y numera ventas
Contraseñas, mails, celulares, pagos	No	No hacen falta para vender y son un riesgo

Esto no es solo prudencia: la **Ley 25.326** de protección de datos personales pide que los datos sean "adecuados, pertinentes y no excesivos" para lo que se usan (art. 4) y que se cuide su seguridad (art. 9). **Lo que no se guarda no se puede filtrar.**

> **Se conecta con lo que ya viste**
>
> En la **Unidad 5** trabajaste mínimo privilegio y datos personales en el servidor. Ahora la misma idea se aplica al teléfono: como SQLite no tiene usuarios ni permisos, **guardar poco es la principal medida de seguridad**.

> **En resumen**
>
> - Al teléfono viaja solo lo necesario para vender.
> - Cada limitación del teléfono se convierte en una decisión de diseño.
> - Nunca viajan contraseñas ni datos personales que no hacen falta.
>
> **Pregunta que queda abierta →** Ya sabemos qué guardar. ¿Cómo lo guardamos bien? → Parada 3

Parada 3 · Guardarlo bien: el esquema de la tablet

> **La pregunta de esta parada**
>
> ¿Cómo armamos las tablas de SQLite para que los datos sean correctos?

> **En el food truck**
>
> La libreta del vendedor tiene **cuatro secciones**: la lista de precios, los pedidos anotados, los productos de cada pedido y una lista de "pendientes de enviar".

De PostgreSQL a SQLite: cómo se traducen los tipos

En PostgreSQL	En SQLite	Por qué
`NUMERIC(10,2)` (precio)	`INTEGER` en centavos	Con decimales de coma flotante, `0.1 + 0.2 = 0.3` da falso. Con centavos, las cuentas son exactas: $1.200 se guarda como 120000
`BOOLEAN`	`INTEGER` con `CHECK (… IN (0,1))`	SQLite no tiene booleano
`ENUM` (forma de pago)	`TEXT` con `CHECK (… IN (…))`	SQLite no tiene enumerados
`TIMESTAMPTZ`	`TEXT` con fecha ISO 8601	SQLite no tiene tipo fecha
`IDENTITY` (id automático)	El id del servidor, o un UUID si el dato nace en la tablet	Sin red no hay quien asigne números

Las cuatro tablas

Tabla	Qué guarda	Analogía
`producto_cache`	Copia del catálogo traída del servidor, con la fecha en que se copió	La lista de precios
`pedido_borrador`	Pedido anotado sin conexión (todavía no es una venta)	El ticket
`item_borrador`	Productos de cada borrador, con el precio que vio el cliente	Los renglones del ticket
`cola_sync`	Cada intento de enviar un borrador, con su resultado	La lista de pendientes

Esquema de la tablet (simplificado del kit: movil/schema_sqlite.sql)

```
PRAGMA foreign_keys = ON;      -- siempre, al abrir la base

CREATE TABLE producto_cache (
    producto_id      INTEGER PRIMARY KEY,              -- el mismo id que en el servidor
    nombre           TEXT    NOT NULL,
    precio_centavos  INTEGER NOT NULL CHECK (precio_centavos >= 0),
    stock_visto      INTEGER NOT NULL CHECK (stock_visto >= 0),   -- solo orientativo
    disponible_visto INTEGER NOT NULL CHECK (disponible_visto IN (0,1)),
    eliminado        INTEGER NOT NULL DEFAULT 0 CHECK (eliminado IN (0,1)),
    capturado_en     TEXT    NOT NULL                  -- cuándo se copió
) STRICT;

CREATE TABLE pedido_borrador (
    id_local            TEXT    PRIMARY KEY,           -- UUID creado en la tablet
    usuario_id_servidor INTEGER NOT NULL,
    forma_pago          TEXT    NOT NULL
                        CHECK (forma_pago IN ('EFECTIVO','TARJETA','TRANSFERENCIA')),
    creado_en           TEXT    NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ','now'))
) STRICT;

CREATE TABLE item_borrador (
    id_local              TEXT    NOT NULL
                          REFERENCES pedido_borrador(id_local) ON DELETE CASCADE,
    producto_id           INTEGER NOT NULL REFERENCES producto_cache(producto_id),
    cantidad              INTEGER NOT NULL CHECK (cantidad > 0),
    precio_centavos_visto INTEGER NOT NULL CHECK (precio_centavos_visto >= 0),
    PRIMARY KEY (id_local, producto_id)
) STRICT;

CREATE TABLE cola_sync (
    solicitud_uuid TEXT PRIMARY KEY,                   -- no cambia si se reintenta
    id_local       TEXT NOT NULL REFERENCES pedido_borrador(id_local),
    estado         TEXT NOT NULL DEFAULT 'PENDIENTE'
                   CHECK (estado IN ('PENDIENTE','CONFLICTO','RECHAZADO','SIMULADO_APROBADO')),
    motivo         TEXT,
    creada_en      TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ','now'))
) STRICT;

-- Regla: un borrador no puede tener dos envíos pendientes al mismo tiempo
CREATE UNIQUE INDEX ux_cola_una_pendiente ON cola_sync (id_local) WHERE estado = 'PENDIENTE';

-- Lo que ve el vendedor en pantalla
CREATE VIEW v_catalogo_disponible AS
```

```
SELECT producto_id, nombre, precio_centavos / 100.0 AS precio, stock_visto
FROM producto_cache
WHERE eliminado = 0 AND disponible_visto = 1;
```

¿Por qué ese índice al final? Un borrador puede enviarse más de una vez en su vida (por ejemplo, si el precio cambió y el cliente acepta el nuevo, se vuelve a enviar). Lo que nunca puede pasar es que tenga **dos envíos pendientes a la vez**. Un índice único que solo mira las filas `PENDIENTE` hace cumplir exactamente esa regla.

> **Se conecta con lo que ya viste**
>
> En la **Unidad 3** usaste `CHECK`, claves foráneas, vistas e índices parciales (como `WHERE eliminado = FALSE`). Acá son los mismos objetos, en otro motor. Y la tabla `item_borrador` repite una idea de Food Store: el **precio congelado** del detalle del pedido.

> **En resumen**
>
> - El precio va en centavos para que las cuentas sean exactas.
> - Cuatro tablas: catálogo, borrador, ítems y cola de envío.
> - Las restricciones (`CHECK`, `FOREIGN KEY`, `STRICT`) cuidan los datos aunque nadie las controle.
>
> **Pregunta que queda abierta →** ¿Y si la app se cierra justo mientras anotamos el pedido? → Parada 4

Parada 4 · Anotar el pedido sin que quede a medias

> **La pregunta de esta parada**
>
> ¿Cómo evitamos que un pedido quede guardado a medias?

> **En el food truck**
>
> El vendedor anota el pedido de Ana: el ticket, la muzzarella, las empanadas y el pendiente de envío. En ese momento la batería se agota. Al prender la tablet, ¿qué quedó guardado?

La respuesta es la de la Unidad 1: una **transacción**. El borrador, sus ítems y su envío pendiente se graban **juntos**: o se guarda el ticket entero, o no se guarda nada. Nunca queda un pedido sin productos ni un pedido que jamás se va a enviar.

Un borrador completo en una sola transacción

```
BEGIN;
INSERT INTO pedido_borrador (id_local, usuario_id_servidor, forma_pago)
VALUES ('aaaaaaaa-0000-4000-8000-000000000001', 2, 'EFECTIVO');

INSERT INTO item_borrador (id_local, producto_id, cantidad, precio_centavos_visto)
VALUES ('aaaaaaaa-0000-4000-8000-000000000001', 3, 2, 120000);     -- 2 empanadas a $1.200

INSERT INTO cola_sync (solicitud_uuid, id_local)
VALUES ('bbbbbbbb-0000-4000-8000-000000000001', 'aaaaaaaa-0000-4000-8000-000000000001');
COMMIT;            -- si algo falla antes de esta línea: ROLLBACK y no queda nada
```

¿Qué es ese código largo? El UUID

En el servidor, el número de pedido lo pone PostgreSQL (1, 2, 3…). Pero la tablet está sin red: si dos tablets numeraran solas, las dos tendrían un "pedido 1". Un **UUID** es un identificador de 36 caracteres generado al azar, que no se repite aunque lo creen dos dispositivos distintos. El número de pedido real lo asigna el servidor cuando confirma la venta. En los ejemplos usamos UUID fáciles de leer; la app los genera al azar.

Atención

Un borrador **no descuenta stock**, **no está pagado** y **no es una venta**. La transacción de SQLite protege la tablet, pero no toca el servidor.

Se conecta con lo que ya viste

Es la **atomicidad** de la Unidad 1 (todo o nada) aplicada a la tablet. Y es la misma idea que `sp_crear_pedido`, que crea el pedido y sus detalles dentro de una transacción en el servidor.

En resumen

- Borrador + ítems + envío pendiente se graban en una sola transacción.
- Lo que nace en la tablet se identifica con un UUID.
- Un borrador no es una venta: no toca el stock del servidor.

Pregunta que queda abierta → El pedido quedó guardado. Pero la lista de precios de la tablet, ¿estará vieja? → Parada 5

Parada 5 · ¿Mi lista está vieja? Instantánea y lápidas

La pregunta de esta parada

¿Cómo sabe la tablet qué datos tiene el servidor, y desde cuándo?

En el food truck

La tablet copió la lista de precios a la mañana. A la tarde, en la oficina, subieron la muzzarella y sacaron la fugazzeta de la carta. La tablet no se enteró: **su lista está vieja**.

Sincronizar es poner de acuerdo la base de la tablet con la del servidor: traer lo que cambió allá y llevar lo que se hizo acá. En esta unidad lo hacemos con dos movimientos:

Movimiento	Qué hace	En Food Store
Bajar (descarga)	Trae una copia actualizada del servidor	La tablet recibe el catálogo con precios, stock y bajas
Subir (envío)	Manda lo que se hizo en la tablet	Los borradores pendientes se envían para validarlos

Ese intercambio puede hacerse al volver la señal, cada cierto tiempo o con un botón “actualizar”. En cualquier caso, la copia del catálogo se llama **instantánea**: es como una lista de precios **con fecha**. Siempre sabemos cuán vieja es.

En PostgreSQL: lo que el servidor le da a la tablet (postgresql/10_snapshot_catalogo.sql)

```
CREATE VIEW v_snapshot_movil AS
SELECT p.id                          AS producto_id,
       p.nombre,
       (p.precio * 100)::INTEGER     AS precio_centavos,   -- la conversión se hace acá, una vez
       p.stock,
       p.disponible,
       (p.eliminado OR c.eliminado) AS eliminado,          -- la baja viaja como dato
       now()                         AS capturado_en       -- la fecha de la lista
FROM producto p
JOIN categoria c ON c.id = p.categoria_id;
```

Las lápidas: tachar, no arrancar la hoja

Si en el servidor se **borrara** la fugazzeta con `DELETE`, la tablet nunca se enteraría: lo que no existe no aparece en la copia. Por eso Food Store usa desde el principio la **baja lógica** (`eliminado = TRUE`): el producto sigue existiendo, marcado como dado de baja. Esa marca viaja a la tablet y se llama **lápida**. La tablet saca el producto de la pantalla, pero conserva la fila, porque algún borrador puede usarla.

Mientras tanto, la tablet y el servidor pueden no coincidir por un rato. Eso se llama **consistencia eventual**: las copias se ponen de acuerdo cuando se sincronizan.

> **Se conecta con lo que ya viste**
>
> La **baja lógica** de Food Store, que parecía un detalle, es lo que hace posible sincronizar bajas. Y la **consistencia eventual** es la que viste en la **Unidad 7** con las bases NoSQL.

> **En resumen**
>
> - Sincronizar es bajar los cambios del servidor y subir lo hecho en la tablet.
> - La instantánea es una copia del catálogo con fecha.
> - Las bajas viajan como lápidas gracias a la baja lógica.
>
> **Pregunta que queda abierta →** Ya tenemos la lista nueva y el borrador viejo. ¿Cómo los comparamos? → Parada 6

Parada 6 · Comparar: el semáforo

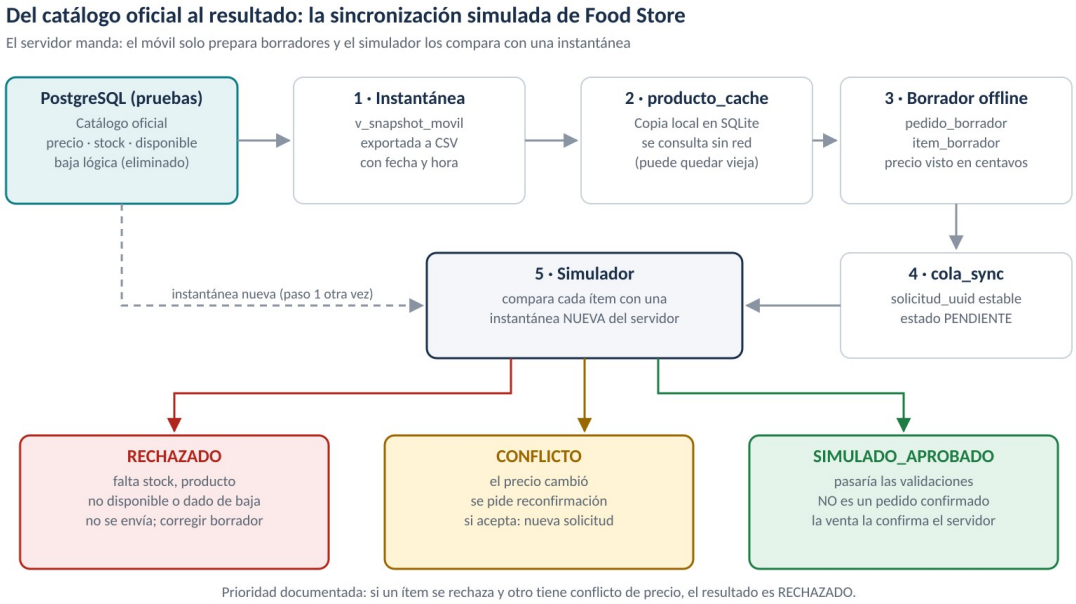
> **La pregunta de esta parada**
>
> Cuando vuelve la señal, ¿cómo decidimos qué pasa con cada borrador?

> **En el food truck**
>
> Vuelve la señal. La tablet tiene el borrador de Ana con la muzzarella a $9.000. La instantánea nueva dice que la muzzarella vale $10.500. Hay que comparar.

El **simulador** toma cada envío pendiente y compara cada producto del borrador con la instantánea nueva del servidor. El resultado es un **semáforo**:

Luz	Estado	Cuándo	Qué se hace
Rojo	`RECHAZADO`	Falta stock, el producto no está disponible o fue dado de baja	No se envía. Se corrige el borrador
Amarillo	`CONFLICTO`	Se puede vender, pero el precio cambió	Se le pregunta al cliente si acepta el precio nuevo
Verde	`SIMULADO_APROBADO`	Todo coincide	Pasaría las validaciones. No es una venta confirmada

Si hay rojo y amarillo en el mismo borrador, gana el rojo. Preguntarle al cliente por el precio no sirve de nada si igual falta un producto.



Del catálogo oficial al semáforo: el recorrido completo de un borrador.

La lógica del simulador, en palabras

```
SIMULAR(solicitud, instantanea_nueva):
    si la solicitud ya fue evaluada: devolver el resultado guardado      (parada 8)
    para cada producto del borrador:
        si no existe, no está disponible o fue dado de baja  -> rojo
        si la cantidad es mayor que el stock                 -> rojo
        si el precio visto no es el precio actual            -> amarillo
    si hubo rojo -> RECHAZADO; si no, si hubo amarillo -> CONFLICTO; si no -> SIMULADO_APROBADO
    guardar estado y motivo en cola_sync
    (no se toca nada en PostgreSQL)
```

Atención

`SIMULADO_APROBADO` quiere decir "con esta lista, el pedido pasaría". Una venta real solo existe cuando el servidor ejecuta `sp_crear_pedido`, que vuelve a revisar el stock con `SELECT … FOR UPDATE`. Entre la instantánea y ese momento, alguien más pudo comprar.

Se conecta con lo que ya viste

El semáforo hace del lado del simulador lo que `sp_crear_pedido` hace en el servidor (Unidad 3): validar disponibilidad y stock. Y el bloqueo `FOR UPDATE` de la **Unidad 1** es lo que, en una venta real, evita que dos tablets se lleven la última unidad.

En resumen

- Cada borrador se compara con una instantánea nueva.
- Rojo (`RECHAZADO`), amarillo (`CONFLICTO`) o verde (`SIMULADO_APROBADO`); el rojo gana.
- Verde no es venta: solo el servidor confirma.

Pregunta que queda abierta → Salió amarillo o rojo. ¿Cómo se resuelve un conflicto? → Parada 7

Parada 7 · Conflictos y cómo se resuelven

La pregunta de esta parada

Cuando la tablet y el servidor no coinciden, ¿quién tiene razón?

En el food truck

Es como **dos personas editando la misma planilla**: cada una cambió algo sin ver lo que hizo la otra. Cuando se juntan, hay que decidir qué queda.

Un **conflicto** aparece cuando lo que hizo la tablet no coincide con lo que dice el servidor. No se puede evitar: se **detecta** y se **resuelve** con una regla pensada de antemano.

Situación	Ejemplo en Food Store	Cómo lo resolvemos
El precio cambió	La tablet tenía $9.000; el servidor, $10.500	Amarillo: se pregunta al cliente. El precio nunca se cambia sin avisar
No alcanza el stock	Se piden 10 empanadas y quedan 4	Rojo: decide el servidor
El producto ya no se vende	La fugazzeta fue dada de baja	Rojo, y la lápida la saca de la tablet
El envío llegó dos veces	Se cortó la señal y se reintentó	Se reconoce por el mismo UUID (parada 8)

Las estrategias posibles

No hay una sola forma de resolver conflictos. Estas son las más comunes (alcanza con conocerlas a nivel de idea):

Estrategia	En una línea	¿Sirve para Food Store?
Gana la última escritura	Vale el cambio más reciente según la hora	No: la hora de los teléfonos no es confiable y se pierden cambios sin avisar
Gana el servidor	Vale lo que dice la base central	Sí, para bajas y precios de lista
Gana el dispositivo	Vale lo que hizo la tablet	No para precio ni stock
Regla del negocio	Decide una regla del dominio	Sí: el stock lo decide el servidor con su regla
Preguntar a una persona	Se informa y alguien decide	Sí: el cliente acepta o no el precio nuevo

¿Cómo se detecta que alguien cambió algo? Una forma muy usada es darle a cada fila un **número de versión**: si la versión que tenía la tablet ya no es la del servidor, hubo un cambio en el medio. No lo pedimos en el TP, pero es la base de muchos sistemas reales.

La historia completa, paso a paso

Momento	En la tablet	En el servidor	Qué pasa
10:00 · con señal	Copia la muzzarella a $9.000	Muzzarella a $9.000	Todo coincide
12:00 · sin señal	Anota el borrador de Ana	Otros clientes compran	El borrador espera
13:00 · sin señal	Sigue con $9.000	Suben la muzzarella a $10.500	Las copias ya no coinciden
14:00 · vuelve la señal	Compara con la instantánea nueva	—	Amarillo: `CONFLICTO`
14:05	Ana acepta $10.500: nuevo envío	—	Verde: `SIMULADO_APROBADO`

> **Se conecta con lo que ya viste**
>
> Es el **lost update** de la **Unidad 1**: dos cambios sobre el mismo dato. Allá se resolvía bloqueando con `FOR UPDATE`; con una tablet desconectada no se puede bloquear nada por horas, así que se detecta después. Y es el **teorema CAP** de la **Unidad 7**: sin red, la tablet elige seguir funcionando (disponibilidad) y se pone de acuerdo después.

> **En resumen**
>
> - Un conflicto se detecta y se resuelve con una regla decidida de antemano.
> - Precio: se pregunta. Stock y bajas: decide el servidor.
> - Nunca se decide por la hora del teléfono.
>
> **Pregunta que queda abierta →** Al reintentar un envío, ¿no se duplica el pedido? → Parada 8

Parada 8 · Si el envío se repite

> **La pregunta de esta parada**
> ¿Cómo evitamos que un pedido se registre dos veces?

> **En el food truck**
> La tablet envía el pedido. El servidor lo recibe… y justo se corta la señal antes de que llegue la respuesta. La tablet no sabe si llegó, así que **reintenta**. ¿Ana termina con dos pedidos?

La solución se llama **idempotencia**: una operación es idempotente si hacerla una vez o diez veces da el mismo resultado. Es como el **botón del ascensor**: apretarlo cinco veces no trae cinco ascensores.

El truco es el **UUID del envío**, que no cambia en los reintentos. En la tablet, el simulador mira si ese envío ya fue evaluado: si es así, devuelve el resultado guardado y no evalúa de nuevo.

Pero en un sistema real, la protección definitiva tiene que estar **en el servidor**: si el servidor no recuerda el UUID, crearía un segundo pedido. Se guarda el UUID con una **clave única**, en la **misma transacción** que crea el pedido:

Cómo sería en producción (lo muestra el docente; el TP no lo pide)

```
CREATE TABLE solicitud_movil (
    solicitud_uuid UUID PRIMARY KEY,         -- el mismo UUID que guarda cola_sync
    usuario_id     BIGINT NOT NULL REFERENCES usuario(id),
    pedido_id      BIGINT REFERENCES pedido(id)
);
-- En el procedimiento del servidor, dentro de la misma transacción que crea el pedido:
INSERT INTO solicitud_movil (solicitud_uuid, usuario_id) VALUES (p_solicitud_uuid, p_usuario_id)
ON CONFLICT (solicitud_uuid) DO NOTHING;     -- si ya existe, no hace nada
IF NOT FOUND THEN RETURN; END IF;            -- ya procesado: no se crea otro pedido
CALL sp_crear_pedido(p_usuario_id, p_forma_pago, p_items);
```

> **Se conecta con lo que ya viste**
> Son dos ideas de la **Unidad 1** juntas: una restricción `UNIQUE` y una **transacción** que hace todo o nada. Si `sp_crear_pedido` falla por falta de stock, también se deshace el registro del UUID, y el envío se puede reintentar.

> **En resumen**
> - Idempotente = repetir no cambia el resultado.
> - El UUID del envío no cambia al reintentar.
> - La protección real contra duplicados está en el servidor: clave única + transacción.
>
> **Pregunta que queda abierta →** Ya resolvimos el pedido de la feria. ¿Cómo encaja todo esto con el resto del proyecto? → Parada 9

Laboratorio de la semana 12

En el laboratorio vas a ver toda la historia funcionando con el **kit de la cátedra**: una base SQLite que hace de tablet, la base PostgreSQL de Food Store y un simulador que reproduce siete casos con verificación automática. **Son los mismos siete casos que tenés que demostrar en el TP**, con tu propia versión de Food Store.

Caso	Situación	Resultado esperado	Parada
C1	Sin señal: consultar el catálogo	Se lee desde SQLite	1 y 3
C2	El precio cambió en el servidor	Amarillo `CONFLICTO`; si el cliente acepta, nuevo envío en verde	6 y 7
C3	No alcanza el stock	Rojo `RECHAZADO`; el stock del servidor no se toca	6 y 7
C4	El producto fue dado de baja	Rojo `RECHAZADO`; la lápida lo saca del catálogo de la tablet	5 y 6
C5	Todo coincide	Verde `SIMULADO_APROBADO`, marcado como simulación	6
C6	El mismo envío llega dos veces	Se devuelve el resultado anterior; no se evalúa de nuevo	8
C7	Dato inválido (cantidad 0 o producto inexistente)	Error controlado; no queda nada guardado	3 y 4

Cómo se ejecuta

Siempre sobre una base de PRUEBAS: --reset la borra y la vuelve a crear

```
pip install psycopg2-binary
cd kit_u8_campus/simulador
python3 simulador_sync.py --dsn "host=localhost dbname=foodstore_u8 user=postgres" --reset
...
Resultado: 16 verificaciones correctas, 0 fallidas.
```

Antes de presentar tu simulación, verificá que

- ☐ `PRAGMA foreign_keys` devuelve 1.
- ☐ No se pueden guardar cantidades en 0 ni productos inexistentes.
- ☐ El catálogo se puede consultar sin conexión al servidor.
- ☐ Cada caso deja un estado y un motivo que se pueden ver.
- ☐ El stock y los pedidos de PostgreSQL no cambian por la simulación.
- ☐ Un envío reintentado conserva su UUID y no se aprueba dos veces.

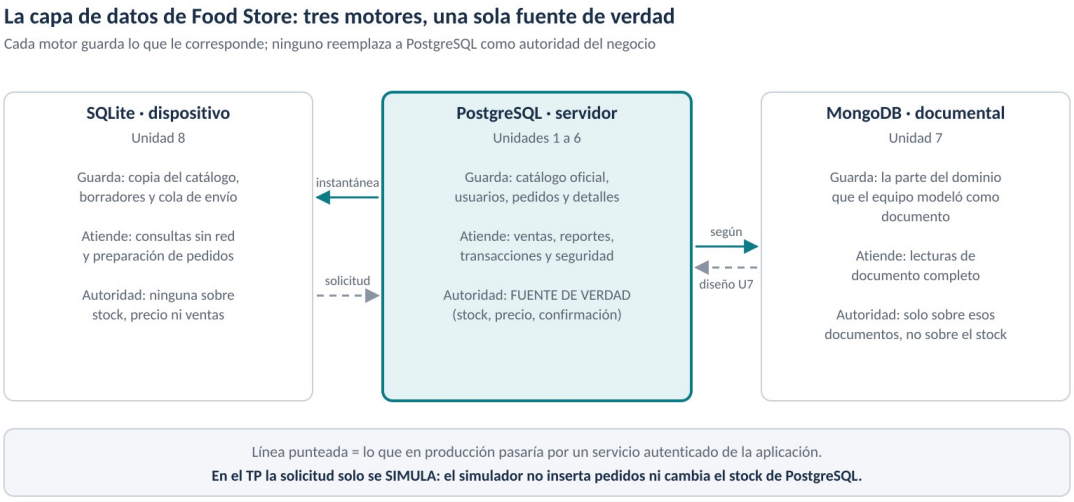
SEMANA 13 · DE LA HISTORIA AL PROYECTO COMPLETO

Parada 9 · Tres motores, cada uno con su oficio

> **La pregunta de esta parada**
>
> ¿Cómo encajan PostgreSQL, MongoDB y SQLite en un mismo proyecto?

> **En el food truck**
>
> Food Store ya no es una sola base: tiene la **oficina central** (PostgreSQL), el **archivo de documentos** que armaste en la Unidad 7 (MongoDB) y **la libreta de cada tablet** (SQLite). Cada una tiene su oficio.

Integrar no es juntar scripts en una carpeta. Es poder decir, para cada dato: **dónde vive, quién lo escribe y quién manda**.



La capa de datos de Food Store: tres motores y una sola fuente de verdad.

Motor	Qué guarda	Quién manda	Lo construiste en
PostgreSQL	Catálogo oficial, usuarios, pedidos, stock	Fuente de verdad: decide precio, stock y ventas	Unidades 1 a 6
MongoDB	La parte del dominio que tu equipo modeló como documento	Solo sobre esos documentos	Unidad 7
SQLite	Copia del catálogo, borradores y cola de envío	Nadie le pide que decida: solo prepara	Unidad 8

El recorrido completo, de la consulta a la venta

1. PostgreSQL publica la instantánea del catálogo.
2. La tablet la guarda y permite armar borradores sin señal.
3. Al volver la señal, se envían los borradores pendientes.
4. PostgreSQL vuelve a validar stock, disponibilidad y precio en una transacción: **solo él confirma la venta**.

5. La tablet guarda la respuesta y actualiza su catálogo.
6. MongoDB, si tu proyecto lo usa, atiende su parte sin tocar el stock ni las ventas.

Dónde se puede romper (y qué unidad lo cuida)

Riesgo	Qué cuidar	Unidad
Dos fuentes de verdad	El stock de la tablet nunca pisa el del servidor	U4 · una sola fuente para cada dato
Datos personales en el teléfono	No copiar contraseñas ni datos que no hacen falta	U5 · seguridad
Restaurar un respaldo viejo	Las tablets quedan con datos “del futuro”: el plan de mantenimiento dice que hay que volver a descargar el catálogo	U6 · mantenimiento
Cambios en el esquema	La base de la tablet tiene versión y se actualiza con el catálogo	U4 · migraciones
Confundir estados	Pendiente, simulado, confirmado y rechazado no son lo mismo	U8

Se conecta con lo que ya viste

Esta parada junta toda la cursada: la base relacional de las **Unidades 1 a 6**, la base documental de la **Unidad 7** y la tablet de la **Unidad 8**. Es lo que vas a defender en la instancia global del proyecto integrador.

En resumen

- Cada motor tiene su oficio; PostgreSQL es la única fuente de verdad.
- Integrar es saber dónde vive cada dato y quién manda.
- El plan de mantenimiento también tiene que pensar en las tablets.

Pregunta que queda abierta → Está todo armado. ¿Cómo sabemos que está bien hecho? → Parada 10

Parada 10 · Revisar con IA y defender

La pregunta de esta parada

¿Cómo revisamos todo lo que hicimos en la cursada y cómo lo defendemos?

En el food truck

Antes de abrir el food truck al público, alguien revisa todo: las tablas, los scripts, los permisos. La IA es un **revisor incansable**, pero **la firma es del equipo**.

El code review en cinco pasos

1. **Congelar:** crear una rama `review-final` en Git.
2. **Revisar por partes:** con OpenCode en el agente **Plan** (analiza sin modificar archivos), una unidad por vez.
3. **Comprobar cada hallazgo en el motor:** una consulta, un `EXPLAIN` o una prueba que falla. Lo que dice la IA no es prueba.
4. **Decidir y anotar cada hallazgo:** aceptado, corregido o descartado, siempre con el motivo. Ninguno queda sin decisión.
5. **Corregir en commits separados** y volver a correr todo desde una base vacía.

Revisá...	Preguntas
Integridad (U1, U3)	¿Se descuenta stock sin validar? ¿Las transacciones se deshacen si algo falla?
Rendimiento (U2, U3)	¿Los índices sirven a consultas que se midieron?
Esquema (U4)	¿Las migraciones conservan los datos y tienen vuelta atrás?
Seguridad (U5)	¿Roles con lo mínimo? ¿Hay contraseñas o datos reales en Git?
Mantenimiento (U6)	¿Hay un respaldo y una restauración probada?
NoSQL (U7)	¿La colección tiene sentido y no duplica la fuente oficial?
Móvil (U8)	¿Las claves foráneas están activas? ¿Cada conflicto queda registrado?

Archivo	Lo que propuso la IA	Qué comprobó el equipo	Decisión
`schema_sqlite.sql`	"Hacé `UNIQUE` la columna `id_local` de la cola"	Con `UNIQUE`, un borrador con conflicto de precio no podría reenviarse	Descartado: se mantiene el índice de una sola pendiente
Simulador	"Usá la hora de la tablet para decidir qué vale"	La hora de los teléfonos no es confiable (parada 7)	Descartado

Ejemplos de cómo se registra. En tu TP tienen que ser hallazgos **de tus propios scripts**. Descartar con fundamento también suma: un review donde se aceptó todo muestra que nadie revisó.

Kiro y OpenCode: del pedido al código

El programa pide que el esquema de la tablet y la sincronización se **especifiquen en Kiro** y se **generen con OpenCode**. El orden es siempre el mismo:

1. **Kiro:** escribís la especificación en tres archivos: `requirements.md` (qué tiene que pasar), `design.md` (cómo se arma) y `tasks.md` (las tareas).
2. **OpenCode:** con el agente **Build** genera los scripts a partir de las tareas.
3. **Vos:** leés cada línea, la probás en una copia y guardás un respaldo antes.
4. **Bitácora:** anotás qué propuso la IA, qué aceptaste y qué descartaste, y por qué.

Los requisitos se escriben en notación **EARS**: **WHEN** *pasa algo* **THE SYSTEM SHALL** *hacer tal cosa*. Cada uno se vincula con el caso que lo prueba:

requirements.md · dos ejemplos

```
REQ-03  WHEN el precio actual es distinto del precio visto
        THE SYSTEM SHALL marcar el envío como CONFLICTO, mostrar los dos precios
        y pedir confirmación.                                  Se prueba con: C2
REQ-05  WHEN se reenvía un envío ya evaluado
        THE SYSTEM SHALL devolver el resultado guardado sin evaluarlo otra vez.
                                                               Se prueba con: C6
```

La defensa

En la defensa cada integrante tiene que poder explicar cualquier script, sin ayuda. La regla de la cátedra es clara: **lo que no se puede explicar se considera no hecho**. En el ensayo de la semana 13 vas a presentar en 15 minutos:

1. El mapa de tu capa de datos: qué vive dónde y quién manda.
2. Una demostración: crear la base de la tablet desde cero y correr un caso que elija la cátedra.
3. Una decisión de diseño y la alternativa que descartaron.
4. Un hallazgo de la IA que aceptaron y uno que descartaron.

> **Se conecta con lo que ya viste**
>
> Es la **política de IA de la cátedra** que usaste en todo el cuatrimestre: la IA propone, vos verificás en el motor y decidís. Y es la preparación para el **segundo parcial**, que se rinde **sin IA**.

> **En resumen**
>
> - La IA revisa; el equipo comprueba en el motor y decide cada hallazgo.
> - Primero el spec en Kiro, después el código con OpenCode, siempre leído y probado.
> - Lo que no se puede explicar en la defensa se considera no hecho.

Repaso: la historia en once líneas

Parada	Lo que te tenés que llevar
0 · El problema	Sin señal hay que vender igual sin perder, duplicar ni vender lo que no hay. El móvil propone, el servidor dispone
1 · SQLite	Base embebida y relacional; activar claves foráneas, usar `STRICT`, escribir de a una operación
2 · Qué viaja	Solo lo necesario para vender; nada de contraseñas ni datos personales que no hacen falta
3 · Esquema	Precio en centavos; cuatro tablas; restricciones que cuidan los datos
4 · Transacción	Borrador + ítems + envío, todo o nada; UUID para lo que nace en la tablet
5 · Instantánea	Copia con fecha; las bajas viajan como lápidas
6 · Semáforo	Rojo `RECHAZADO`, amarillo `CONFLICTO`, verde `SIMULADO_APROBADO`; el rojo gana; verde no es venta
7 · Conflictos	Precio: se pregunta. Stock y bajas: decide el servidor. Nunca la hora del teléfono
8 · Idempotencia	Repetir no duplica; la protección real es una clave única en el servidor
9 · Integración	Tres motores, una fuente de verdad (PostgreSQL)
10 · Review y defensa	La IA propone; el equipo verifica, decide y defiende

Autoevaluación para el segundo parcial

El segundo parcial (semana 14) es presencial, individual y sin IA, y toma las unidades 4 a 8. Respondé por escrito antes de mirar las respuestas.

1. ¿Qué es una base embebida? Nombrá dos diferencias con PostgreSQL.
2. En una tablet se guardó un ítem de un producto que no existe y SQLite no dio error. ¿Por qué pudo pasar?
3. ¿Por qué el precio se guarda en centavos?
4. ¿Por qué el borrador, sus ítems y su envío se guardan en una sola transacción?
5. ¿Qué es una lápida y por qué depende de la baja lógica?
6. Un borrador tiene la muzzarella a $9.000 y el servidor dice $10.500. ¿Qué color da el semáforo y qué sigue?
7. Un borrador tiene un producto sin stock y otro con el precio cambiado. ¿Qué resultado da?
8. ¿Qué significa que una operación sea idempotente? ¿Dónde tiene que estar la protección real?
9. ¿Por qué no se usa la hora del teléfono para decidir qué cambio vale?
10. ¿Qué guarda cada motor de Food Store y cuál es la fuente de verdad?

Respuestas

1. Una base que vive dentro de la app y es un archivo. Diferencias: no tiene servidor ni usuarios, escribe de a una operación, funciona sin red, tiene las claves foráneas apagadas por defecto.
2. Porque no se ejecutó `PRAGMA foreign_keys = ON` al abrir la base.

3. Porque los decimales de coma flotante tienen errores de redondeo; con enteros las cuentas son exactas.
4. Para que, si la app se cierra a mitad, no quede un pedido incompleto o que nunca se envíe: todo o nada.
5. Es la marca de “dado de baja” que viaja a la tablet. Si el producto se borrara con `DELETE`, la tablet nunca se enteraría.
6. Amarillo (`CONFLICTO`). Se le pregunta al cliente: si acepta, se actualiza el borrador y se hace un envío nuevo.
7. Rojo (`RECHAZADO`): el rojo gana, porque aceptar el precio no resuelve la falta de stock.
8. Que repetirla no cambia el resultado. La protección real está en el servidor: una clave única por envío, en la misma transacción que crea el pedido.
9. Porque los relojes de los teléfonos pueden estar mal o cambiarse a mano.
10. PostgreSQL: catálogo, usuarios, pedidos y stock (fuente de verdad). MongoDB: la parte documental. SQLite: copia del catálogo y borradores.

Glosario

Palabra	Qué significa	Parada
Base embebida	Base de datos que vive dentro de la app, como un archivo	1
`STRICT`	Tabla de SQLite que controla los tipos de datos	1
Centavos	Forma de guardar dinero como número entero, sin errores de redondeo	3
Borrador	Pedido anotado en la tablet; todavía no es una venta	3 y 4
Cola de envío (`cola_sync`)	Lista de borradores pendientes de enviar, con su resultado	3
UUID	Identificador de 36 caracteres que se crea sin servidor y no se repite	4
Sincronizar	Poner de acuerdo la base de la tablet con la del servidor	5
Instantánea	Copia del catálogo del servidor, con la fecha en que se tomó	5
Lápida	Marca de “dado de baja” que viaja a la tablet	5
Consistencia eventual	Las copias pueden diferir un rato, pero se ponen de acuerdo al sincronizar	5
Conflicto	Lo que hizo la tablet no coincide con lo que dice el servidor	7
Idempotencia	Repetir una operación no cambia el resultado	8
Fuente de verdad	La base que decide cuando hay diferencias: PostgreSQL	9
Code review	Revisión de los scripts, con decisión documentada para cada hallazgo	10
EARS	Forma de escribir requisitos: WHEN … THE SYSTEM SHALL …	10

Bibliografía

Bibliografía del programa de Base de Datos II y documentación oficial consultada en octubre de 2026.

- Programa de Base de Datos II, ciclo lectivo 2026 (Unidad 8, cronograma y política de uso de IA). Proyecto Food Store de la cátedra.
- Elmasri, R.; Navathe, S. *Fundamentos de Sistemas de Bases de Datos*. Pearson (transacciones y concurrencia).
- Silberschatz, A.; Korth, H.; Sudarshan, S. *Fundamentos de Bases de Datos*. McGraw-Hill.
- Sadalage, P.; Fowler, M. *NoSQL Distilled*. Addison-Wesley (consistencia y versiones).
- SQLite: https://www.sqlite.org/whentouse.html · https://www.sqlite.org/foreignkeys.html · https://www.sqlite.org/datatype3.html · https://www.sqlite.org/stricttables.html
- PostgreSQL 16: https://www.postgresql.org/docs/16/
- MongoDB: https://www.mongodb.com/docs/manual/data-modeling/embedding/
- Android Room: https://developer.android.com/training/data-storage/room · Apple Core Data: https://developer.apple.com/documentation/coredata
- Kiro: https://kiro.dev/docs/specs/concepts · OpenCode: https://opencode.ai/docs/agents/
- Chacon, S.; Straub, B. *Pro Git*: https://git-scm.com/book
- Mollick, E. *Co-Intelligence: Living and Working with AI*. Portfolio.
- OWASP. *SQL Injection Prevention Cheat Sheet*: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- Ley 25.326 de Protección de los Datos Personales: https://www.argentina.gob.ar/normativa/nacional/64790/texto